DID Case-Literature Companion Resource

University Review Brief

Prepared: 2026-04-09
Project lead: Shogo Kajimura
Project scope: multilingual DID/MPD case-literature review, restricted OCR support, and public-safe evidence-map companion site

1. Purpose

This document summarizes the current architecture and operating model of the DID case-literature companion resource for internal university review.

The project has two linked but intentionally separated functions:

1. a public-facing companion website that exposes only public-safe metadata, coded variables, figures, and versioned snapshot downloads
2. a restricted submission and OCR workflow that allows external contributors to upload PDFs for OCR processing and later research use

The service is intended to support:

- multilingual evidence mapping of DID/MPD case literature
- OCR processing of scanned or hard-to-access case reports
- later review and synthesis of DID-relevant literature
- possible future meta-analytic or structured review updates

It is not intended to publicly redistribute copyrighted full-text PDFs.

2. Public URLs and Current Deployment

Public site

- Public explorer root:
- https://shogokajimura.github.io/did-case-explorer/
- Public submission page:
- https://shogokajimura.github.io/did-case-explorer/submit.html
- Public status page:
- https://shogokajimura.github.io/did-case-explorer/status.html

Restricted backend

- OCR backend host:
- local Mac operated by the project lead
- External exposure:
- Tailscale Funnel
- Current backend origin:
- https://shogomac-studio.taila49884.ts.net/

The restricted backend is not a general public repository. It only accepts uploads through the submission workflow and serves private job status and OCR artifacts to the uploader via tokenized status links.

3. High-Level System Structure

Public layer

- Host: GitHub Pages
- Files: static HTML, CSS, JavaScript, JSON, TSV, and documentation
- Function:
- public browsing of reviewed metadata
- public-safe figures and tables
- public snapshot downloads
- submission form UI
- private status page UI

Restricted backend layer

- Host: local Mac
- API: FastAPI
- OCR runtime:
- Tesseract
- pdftoppm / poppler
- pdfunite
- Execution model:
- local API process
- local worker process
- launchd auto-restart

External access layer

- Tailscale Funnel exposes only the restricted API endpoint over HTTPS
- Cloudflare Turnstile protects the public submission form from automated abuse

4. End-to-End Data Flow

Public snapshot flow

1. reviewed literature metadata and coded variables are prepared locally
2. a versioned public-safe export is generated
3. the static explorer is updated and deployed to GitHub Pages
4. users can browse only public-safe outputs

Restricted submission and OCR flow

1. an external user opens the public submission page
2. the user uploads one PDF and confirms lawful access
3. Cloudflare Turnstile performs human-verification
4. the browser sends the file to the restricted backend endpoint
5. the backend creates a job record and stores the PDF locally
6. the backend returns a private status link with an access token
7. the local OCR worker processes the PDF
8. the worker produces:
- searchable.pdf
- canonical.txt
9. the worker runs a DID-relevance screen on OCR text and supplied metadata

10. if the result is clearly DID-relevant or probably relevant, artifacts can be downloaded from the private status page
11. if the OCR result appears unrelated, download is temporarily held and bibliographic clarification is requested
12. only after later audit can metadata from the submission enter a future public corpus update

5. Public vs Restricted Data Boundary

Publicly exposed

- bibliographic metadata approved for release
- coded clinical variables approved for release
- source-access labels
- public-domain or clearly open links
- aggregate figures and downloadable public snapshot files
- contributor names only if the person explicitly enters a public display name

Restricted / not publicly exposed

- uploaded PDFs
- copyrighted source files
- OCR return artifacts for restricted documents
- private contributor identifiers unless intentionally supplied for public credit
- submitter email and affiliation
- queue state and internal audit notes
- internal local file paths

6. Submission and Access Controls

Controls currently implemented

- PDF-only upload acceptance
- file signature check for PDF uploads
- lawful-access checkbox
- hidden honeypot field
- per-client submission rate limiting
- Cloudflare Turnstile human-verification
- token-protected private status links
- no public listing of uploaded files
- no public browsing of restricted OCR artifacts
- Cache-Control: no-store on status and artifact endpoints

Current relevance screen

After OCR completes, the system screens text for DID/MPD relevance.

Possible states:

- likely_relevant
- possible_relevant
- needs_information

If a file appears unrelated to DID/MPD literature, the system does not

currently hard-delete it automatically. Instead:

- download is held
- the status page requests additional bibliographic clarification
- later manual or semi-manual review can determine whether to keep or exclude the record

This design is intended to reduce false negatives for non-English or historically scanned material.

7. OCR Quality and Language Handling

The OCR pipeline is optimized for quality rather than maximal speed.

Current quality strategy

- page rasterization from PDF
- searchable PDF generation
- canonical text extraction
- script-aware automatic language selection when the uploader does not specify OCR language

Examples:

- Hebrew-first documents are routed toward heb+eng
- Japanese-first documents are routed toward jpn+eng
- Korean-first documents are routed toward kor+eng
- Han script documents are routed toward Chinese-plus-English bundles

This was introduced because generic multilingual defaults produced lower-quality OCR for certain non-Latin scripts.

Current processing model

- one worker process is used for stable queue handling
- queue position and jobs ahead are visible on the private status page
- the architecture could later be extended to safe parallel workers, but current live operation prioritizes reliability and output quality

8. Local Runtime and Persistence

Backend runtime

- API process: FastAPI + uvicorn
- worker process: local Python worker
- process supervision: launchd
- external access: Tailscale Funnel

Current assumption

The service remains available while:

- the Mac is powered on
- the user session is active
- launchd agents are running
- Tailscale Funnel is active

Current storage location

- local application support directory under the project lead's user account
- job records, uploaded PDFs, and OCR outputs are stored locally on that machine

9. Versioning and Release Discipline

The public explorer is intentionally versioned by manuscript or review freeze.

Current public release model:

- public explorer reflects a frozen manuscript-aligned snapshot dated 2026-03-27
- new uploads are treated as post-freeze contributions
- post-freeze contributions do not automatically enter the public dataset
- later public updates require metadata and legal review

This separation is important for reproducibility and for legal boundary control.

10. Ethical and Legal Review Questions for the University

The following points require institutional confirmation.

Copyright / intellectual property

- whether restricted OCR processing of contributor-submitted PDFs is acceptable for this research purpose
- whether returning OCR-derived files to the original submitter is permissible
- whether additional terms of use, notices, or takedown procedures are required

Personal information

- whether submitter name, email, and affiliation may be collected under the current workflow
- what privacy notice or retention statement is required
- whether contributor opt-in display is sufficient for public listing of names and links

Research compliance

- whether this service falls within ordinary literature-review infrastructure or requires separate institutional review
- whether any ethics-office or research-compliance filing is needed because external contributors submit files and identifying information

Institutional IT / operational approval

- whether a university-affiliated local machine may be used as a restricted backend
- whether an externally exposed Tailscale Funnel endpoint is acceptable
- whether there are required security baselines, logging rules, or

incident-report procedures

11. Current Limitations

- restricted backend currently runs on a local Mac rather than managed university infrastructure
- full long-term retention and deletion policy is not yet formalized
- automated relevance screening is conservative but not perfect
- the service is intended for small-scale usage at present and is not yet a fully managed institutional platform

12. Planned Next-Step Improvements

- formal usage policy and privacy notice
- explicit retention and deletion schedule
- release log distinguishing manuscript-freeze records from later audited additions
- safer multi-worker queue design if volume grows
- possible migration of the restricted backend from local hosting to managed infrastructure

13. Summary For Administrative Review

In plain terms, the current system works as follows:

- the public website is a static GitHub Pages site
- users can submit PDFs from that site
- the PDF is not stored on GitHub Pages
- the PDF is sent to a restricted backend on a local machine
- the local machine performs OCR and prepares downloadable files
- the uploader receives access only through a tokenized private status link
- public release is limited to reviewed metadata and aggregate outputs

The key institutional questions are therefore not only about the public

website, but also about:

- OCR processing of contributed PDFs
- return of OCR-derived files to contributors
- collection of submitter information
- operation of a university-affiliated restricted backend exposed to the internet